

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG KHOA CÔNG
NGHỆ THÔNG TIN



HỆ THỐNG CHATBOT TRỢ GIẢNG **THÔNG MINH**

BÁO CÁO CUỐI KỲ **PHÁT TRIỂN PHẦN MỀM FULL-STACK**

Người hướng dẫn: **THS. VŨ ĐÌNH HỒNG**

Người thực hiện: Nguyễn Văn Phát - 52300049

Hoàng Minh Quân - 52300245

Nguyễn Thành Thắng - 52300256

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG KHOA CÔNG
NGHỆ THÔNG TIN



HỆ THỐNG CHATBOT TRỢ GIẢNG **THÔNG MINH**

BÁO CÁO CUỐI KỲ **PHÁT TRIỂN PHẦN MỀM FULL-STACK**

Người hướng dẫn: **THS. VŨ ĐÌNH HỒNG**

Người thực hiện: Nguyễn Văn Phát - 52300049

Hoàng Minh Quân - 52300245

Nguyễn Thành Thắng - 52300256

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành bày tỏ sự biết ơn sâu sắc và lòng kính trọng đến thầy **Vũ Đình Hồng**, thầy là người hướng dẫn chúng em trong báo cáo lần này. Trong quá trình học tập và làm việc, thầy đã truyền đạt cho chúng em vô vàn kiến thức hay và bổ ích, giúp chúng em có được cơ sở lý thuyết vững vàng để chúng em có thể hoàn thành dự án này.

Tuy nhiên, vì sự hiểu biết còn hạn chế của chúng em, bài báo cáo còn nhiều sai sót và chưa chính chu như chúng em mong muốn. Chúng em rất mong nhận được nhận xét và đánh giá của thầy cô để có thể rút kinh nghiệm cũng như sửa chữa lỗi sai của bản thân.

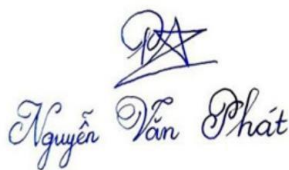
Chúng em xin kính chúc quý thầy, quý cô và quý nhà trường luôn mạnh khỏe, hạnh phúc và ngày một thành công hơn trong sự nghiệp trồng người của mình.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 19 tháng 04 năm 2026

Tác giả

(Ký và ghi rõ họ tên)



Nguyễn Văn Phát



Nguyễn Thành Thắng



Hoàng Minh Quân

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Nhóm chúng em xin cam đoan đây là công trình nghiên cứu của riêng chúng em và được sự hướng dẫn khoa học của thầy **Vũ Đình Hồng**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

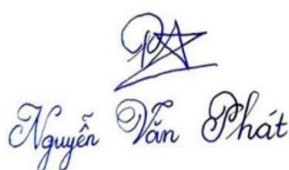
Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào nhóm chúng em xin hoàn toàn chịu trách nhiệm về nội dung Báo cáo thực hành của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng em gây ra trong quá trình thực hiện (nếu có).

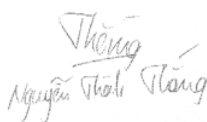
TP. Hồ Chí Minh, ngày 19 tháng 04 năm 2026

Tác giả

(Ký và ghi rõ họ tên)



Nguyễn Văn Phát



Nguyễn Thành Thắng



Hoàng Minh Quân

PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIÁO VIÊN

Phần xác nhận của giảng viên hướng dẫn

TP. Hồ Chí Minh, ngày .. tháng .. năm 2026

(Chữ ký và Họ tên của Giảng viên)

Phần đánh giá của giảng viên chấm bài

TP. Hồ Chí Minh, ngày .. tháng .. năm 2026

(Chữ ký và Họ tên của Giảng viên)

TÓM TẮT ĐỀ TÀI

Đề án xây dựng hệ thống Chatbot Trợ giảng Thông minh nhằm giải quyết vấn đề hallucination của các mô hình ngôn ngữ lớn trong giáo dục đại học. Hệ thống sử dụng RAG kết hợp với mô hình Gemini để đảm bảo chatbot chỉ trả lời dựa trên tài liệu do giảng viên cung cấp.

Về kỹ thuật, ứng dụng được xây dựng theo kiến trúc Full-Stack với Frontend ReactJS và Backend Django. Tài liệu PDF được xử lý qua bước chunking và embedding, sau đó lưu vào cơ sở dữ liệu vector ChromaDB để phục vụ truy xuất ngữ cảnh. Hệ thống hỗ trợ phản hồi thời gian thực qua SSE, giao diện Expansive Layout và tính năng tạo câu hỏi trắc nghiệm giúp sinh viên tự ôn tập.

Kết quả thực nghiệm cho thấy hệ thống truy xuất đúng nội dung từ giáo trình, phản hồi chính xác trong phần lớn các câu hỏi thuộc phạm vi tài liệu và từ chối trả lời khi câu hỏi nằm ngoài nội dung môn học. Toàn bộ hệ thống được đóng gói bằng Docker, sẵn sàng cho việc triển khai thực tế.

MỤC LỤC

DANH MỤC HÌNH VẼ	vii
CHƯƠNG 1 MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI	1
1.1 Đặt vấn đề và Lý do chọn đề tài:	1
1.1.1 Hiện trạng ứng dụng AI trong giáo dục đại học:	1
1.1.2 Nhu cầu tự học và vấn đề Ảo giác thông tin (Hallucination):	1
1.2 Mục tiêu của đồ án:	1
1.3 Phạm vi và Đối tượng nghiên cứu:	1
1.4 Các công nghệ sử dụng trong dự án:	2
CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	3
2.1 Phân tích Use Case và Luồng nghiệp vụ chính	3
2.1.1 Sơ đồ Use Case:	3
2.1.2 Luồng tác vụ của Quản trị viên và Giảng viên:	3
2.1.3 Luồng tác vụ của Sinh viên:	4
2.2 Thiết kế Cơ sở dữ liệu (Database Design):	4
2.2.1 Sơ đồ ERD và Django ORM:	4
2.2.2 Tích hợp ChromaDB (Vector Database):	5
2.2.3 Sơ đồ Lớp (Class Diagram):	6
CHƯƠNG 3 KIẾN TRÚC HỆ THỐNG VÀ CÔNG NGHỆ RAG	8
3.1 Sơ đồ luồng dữ liệu RAG (RAG Pipeline):	8
3.2 Cơ chế phân vùng dữ liệu (Collection Isolation):	9
CHƯƠNG 4 GIẢI THÍCH KỸ THUẬT CHUYÊN SÂU	10
4.1 Kỹ thuật phân đoạn tài liệu:	10
4.2 Thiết kế Prompt (Prompt Engineering) để ngăn chặn Hallucination:	10
4.3 Phản hồi thời gian thực bằng Server-Sent Events (SSE):	11
4.4 Thực chứng kiểm thử Ảo giác thông tin (Hallucination Case Studies)	11
CHƯƠNG 5 GIAO DIỆN (UI) VÀ TRẢI NGHIỆM TƯƠNG TÁC (UX)	14
5.1 Bố cục toàn màn hình (Expansive Layout):	15
5.2 Cơ chế thanh công cụ động (Collapsible Sidebar):	16
5.3 Trang cài đặt dạng Split-View:	17

5.4	Hệ thống Trắc nghiệm sinh động (Gamification)	18
CHƯƠNG 6 TRIỂN KHAI VÀ ĐÓNG GÓI HỆ THỐNG BẰNG DOCKER.....		19
6.1	Container hóa môi trường (Containerization):.....	19
6.2	Cấu hình mạng nội bộ (Bridge Network) với Docker Compose:	19
6.3	Khởi chạy toàn bộ hệ thống bằng một lệnh duy nhất:	19
CHƯƠNG 7 TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN		20
7.1	Đánh giá kết quả ứng dụng	20
7.2	Hạn chế còn tồn đọng của hệ thống	20
7.3	Hướng phát triển chuyên môn sâu trong tương lai	20
PHỤ LỤC: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM.....		21
TÀI LIỆU THAM KHẢO		22

DANH MỤC HÌNH VẼ

Hình 1 Usecase Diagram	3
Hình 2 Erd Diagram.....	4
Hình 3 Class Diagram	6
Hình 4 Sequence Diagram	8
Hình 5 Giao diện Chatbot phản hồi kiến thức từ tài liệu môn học.	12
Hình 6 Giao diện Chatbot phản hồi Câu hỏi không liên quan/không có trong tài liệu	13
Hình 7 Một số mẫu Giao diện Figma đã triển khai(1).....	14
Hình 8 Một số mẫu Giao diện Figma đã triển khai(2).....	15
Hình 9 Giao diện bố cục mở rộng.....	16
Hình 10 Sidebar đóng	16
Hình 11 Sidebar Mở.....	16
Hình 12 Giao diện cài đặt người dùng.	17
Hình 13 Giao diện chức năng luyện tập trắc nghiệm (sai).	18
Hình 14 Giao diện chức năng luyện tập trắc nghiệm (đúng).....	18

CHƯƠNG 1 MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề và lý do chọn đề tài:

1.1.1 Hiện trạng ứng dụng AI trong giáo dục đại học:

Các mô hình ngôn ngữ lớn như ChatGPT hay Gemini đang thay đổi cách sinh viên tìm kiếm và xử lý thông tin. Sinh viên ngày càng có xu hướng dùng AI để tra cứu kiến thức và giải thích các khái niệm phức tạp ngoài giờ học. Tuy nhiên, việc dùng trực tiếp các mô hình AI thương mại bộc lộ một hạn chế nghiêm trọng: chúng không đảm bảo tính chính xác của nội dung học thuật chuyên ngành.

1.1.2 Nhu cầu tự học và vấn đề Ảo giác thông tin (Hallucination):

Dữ liệu huấn luyện của các LLM là cố định và được thu thập rộng rãi từ Internet. Khi sinh viên hỏi về nội dung cụ thể của một môn học, chẳng hạn cấu trúc bài giảng hay tiêu chí đánh giá do giảng viên biên soạn, mô hình không có sẵn dữ liệu đó. Thay vì từ chối, chúng thường ghép thông tin từ nhiều nguồn khác nhau để tạo ra câu trả lời "có vẻ hợp lý" nhưng thực chất sai so với nội dung môn học. Hiện tượng này gọi là hallucination, và đây là rủi ro lớn khi AI truyền đạt kiến thức sai lệch đến sinh viên.

Để giải quyết vấn đề này, đồ án xây dựng hệ thống Chatbot Trợ giảng Thông minh, ứng dụng kỹ thuật RAG để giới hạn phạm vi tra cứu của AI chỉ trong tài liệu nội bộ do giảng viên cung cấp, từ đó đảm bảo độ chính xác của thông tin.

1.2 Mục tiêu của đồ án:

- Xây dựng nền tảng web Full-Stack hoàn chỉnh gồm Front-end, Back-end và hệ thống lưu trữ đa dạng (cơ sở dữ liệu quan hệ và phi quan hệ).
- Ứng dụng kỹ thuật RAG để kiểm soát nguồn tri thức: tích hợp quy trình nhập tài liệu, chuyển đổi văn bản thành vector, giúp Gemini chỉ sử dụng tài liệu được giảng viên phê duyệt khi trả lời. Hệ thống cũng tự động trích dẫn nguồn tài liệu tham chiếu trong mỗi câu trả lời.

1.3 Phạm vi và Đối tượng nghiên cứu:

- Phạm vi hệ thống: Tập trung vào các môn học công nghệ. Tài liệu đầu vào giới hạn ở định dạng PDF để đảm bảo tính nhất quán khi trích xuất văn bản.
- Đối tượng người dùng: Hệ thống phân chia ba vai trò với quyền truy cập riêng biệt:
 - ♦ Quản trị viên: Quản lý và điều phối hệ thống.
 - ♦ Giảng viên: Tải lên và phê duyệt tài liệu học thuật.
 - ♦ Sinh viên: Sử dụng chức năng hỏi đáp AI.

1.4 Các công nghệ sử dụng trong dự án:

Hệ thống được xây dựng theo kiến trúc đa tầng với các công nghệ sau:

1. Front-end: ReactJS, hỗ trợ kiến trúc component và Single Page Application.
2. Back-end: Python kết hợp Django framework. Python được chọn nhờ khả năng tích hợp tốt với các thư viện Machine Learning, đặc biệt là LangChain để xử lý LLM.
3. Cơ sở dữ liệu:
 - ♦ PostgreSQL: Lưu trữ dữ liệu người dùng, môn học và lịch sử hội thoại.
 - ♦ ChromaDB (VectorDB): Lưu trữ và tìm kiếm dữ liệu dạng vector phục vụ kỹ thuật RAG.

Tầng	Công nghệ	Vai trò
Frontend	ReactJS + Vite	Giao diện người dùng (SPA)
Backend	Python + Django	Xử lý logic, REST API
AI/RAG	LangChain + Gemini	Xử lý ngôn ngữ, sinh câu trả lời
Vector DB	ChromaDB	Lưu trữ và tìm kiếm vector
Relational DB	PostgreSQL	Lưu dữ liệu người dùng, môn học
DevOps	Docker Compose	Đóng gói và triển khai

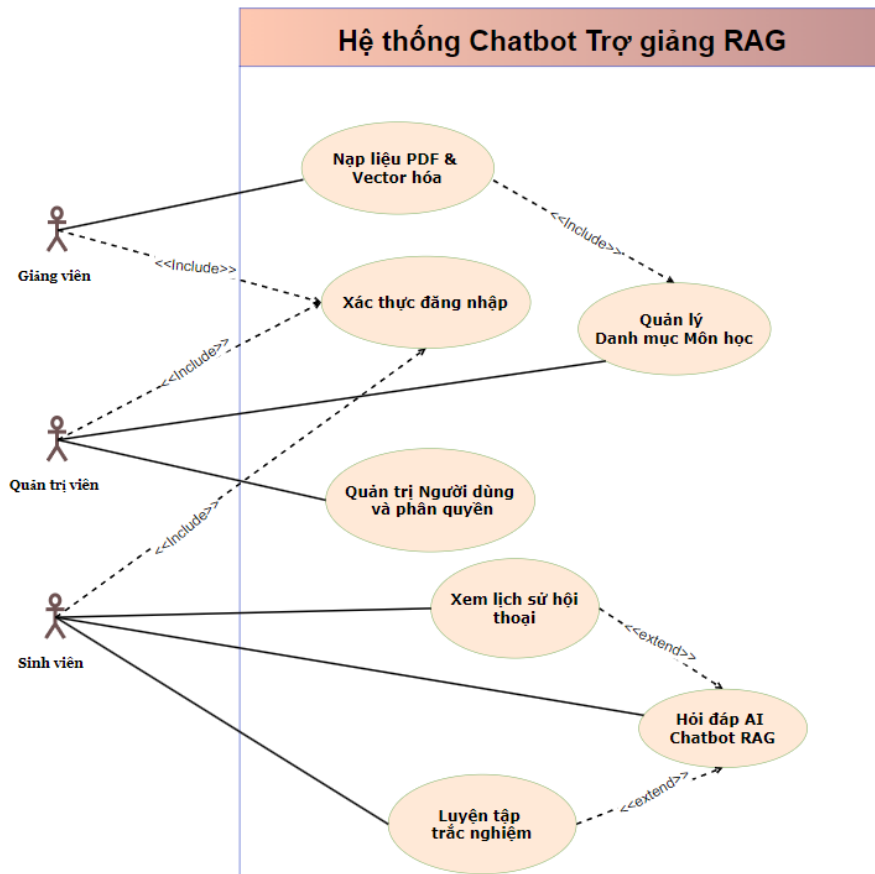
CHƯƠNG 2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích Usecase và luồng nghiệp vụ chính

Phân tích hệ thống là bước quan trọng để xác định các chức năng cần có, từ đó đáp ứng đúng nhu cầu của từng nhóm người dùng.

2.1.1 Sơ đồ Usecase:

Hệ thống chia thành hai khối chức năng chính: khối quản trị (Admin, Giảng viên) và khối học tập (Sinh viên).



Hình 1 Usecase Diagram

2.1.2 Luồng tác vụ của quản trị viên và giảng viên:

- Quản trị viên: Quản lý tài khoản người dùng, phân quyền vai trò và tạo các môn học trong hệ thống.
- Giảng viên: Sau khi đăng nhập, giảng viên truy cập Dashboard để quản lý tài liệu học phần. Khi tải lên file PDF, hệ thống tự động xử lý nền, bóc tách nội dung và chuyển đổi thành dạng vector, để phục vụ tính năng chatbot.

2.1.3 Luồng tác vụ của Sinh viên:

Đây là nhóm người dùng trực tiếp hưởng lợi từ hệ thống. Sau khi đăng nhập, sinh viên có thể:

- Hỏi đáp với AI Chatbot: Mỗi câu hỏi được hệ thống tìm kiếm trong kho tài liệu vector tương ứng với môn học, đảm bảo câu trả lời bám sát nội dung giảng viên đã tải lên.
- Làm bài trắc nghiệm: Hệ thống tạo câu hỏi trắc nghiệm tự động từ nội dung tài liệu, giúp sinh viên ôn tập và tự kiểm tra kiến thức ngay trên giao diện chat.
- Xem lại lịch sử hội thoại: Toàn bộ tin nhắn trong mỗi phiên trò chuyện đều được hệ thống lưu trữ vào cơ sở dữ liệu thông qua model ChatSession và Message. Khi sinh viên đăng nhập lại hoặc quay lại môn học, hệ thống tự động tải lại toàn bộ lịch sử hội thoại trước đó và có thể tiếp tục học tập mà không mất ngữ cảnh.

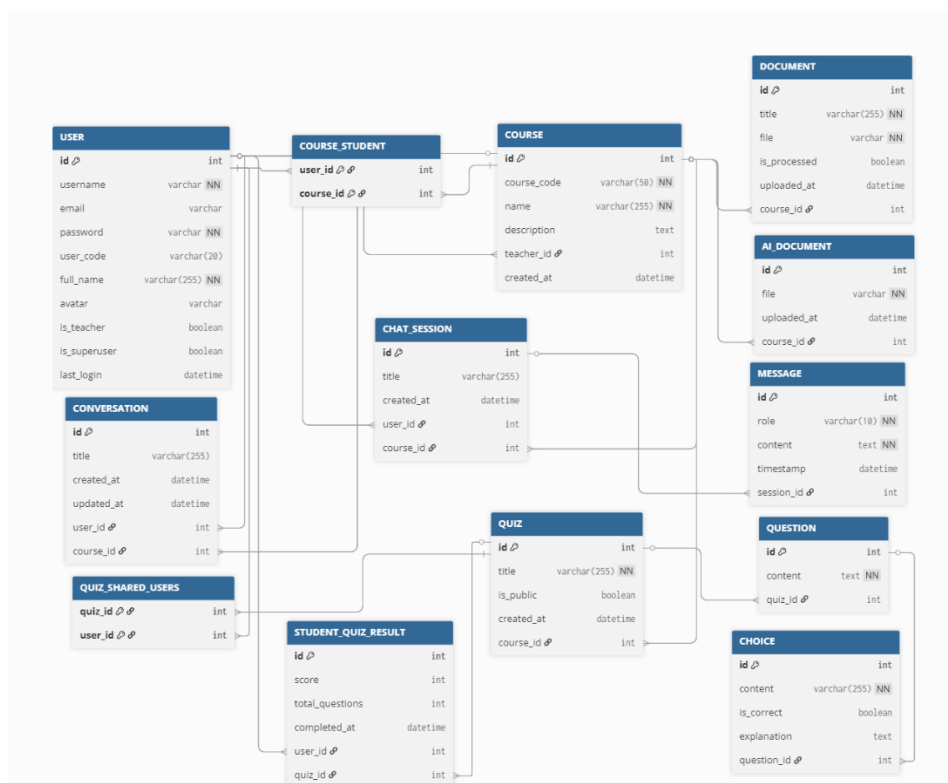
2.2 Thiết kế cơ sở dữ liệu:

Hệ thống dùng song song hai loại cơ sở dữ liệu: cơ sở dữ liệu quan hệ để lưu thông tin người dùng, môn học, tài liệu và ChromaDB để phục vụ tính năng RAG.

2.2.1 Sơ đồ ERD và Django ORM:

Cơ sở dữ liệu quan hệ được thiết kế theo chuẩn 3NF và quản lý thông qua Django ORM.

Hệ thống gồm 13 bảng, chia thành 3 nhóm chính:



Hình 2 Erd Diagram

- Nhóm quản lý người dùng và môn học:
 - ♦ User: Lưu thông tin tài khoản gồm username, email, mã số sinh viên/giảng viên (user_code), họ tên và ảnh đại diện. Trường is_teacher phân biệt giảng viên và sinh viên, trường is_superuser đánh dấu quản trị viên.
 - ♦ Course: Lưu thông tin môn học gồm mã môn và tên. Mỗi môn liên kết với một giảng viên qua ForeignKey (SET_NULL, giảng viên nghỉ thì môn vẫn còn). Sinh viên ghi danh qua bảng trung gian Course_Student (quan hệ nhiều-nhiều).
 - ♦ Document: Lưu tài liệu PDF theo từng môn học. Trường is_processed cho biết tài liệu đã được vector hóa hay chưa. Khi xóa môn học, tài liệu tự động xóa theo (CASCADE).
- Nhóm hội thoại:
 - ♦ ChatSession: Lưu phiên trò chuyện, liên kết với User và Course. Mỗi phiên có tiêu đề và thời gian tạo.
 - ♦ Message: Lưu từng lượt hỏi-đáp trong phiên chat. Trường role nhận giá trị user hoặc ai để phân biệt người gửi.
 - ♦ Conversation: Bảng quản lý cuộc trò chuyện ở mức cao hơn, có thêm trường updated_at để sắp xếp theo lần cập nhật mới nhất.
 - ♦ AIDocument: Lưu tài liệu tri thức AI riêng theo từng môn, tách biệt với tài liệu PDF thông thường.
- Nhóm trắc nghiệm:
 - ♦ Quiz: Lưu bài trắc nghiệm theo môn học. Trường is_public cho phép chia sẻ công khai, ngoài ra có bảng trung gian Quiz_Shared_Users để chia sẻ riêng cho từng sinh viên.
 - ♦ Question: Lưu câu hỏi thuộc một bài quiz.
 - ♦ Choice: Lưu các đáp án cho từng câu hỏi, gồm nội dung, trạng thái đúng/sai và giải thích.
 - ♦ StudentQuizResult: Lưu kết quả làm bài của sinh viên gồm điểm số và tổng số câu hỏi.

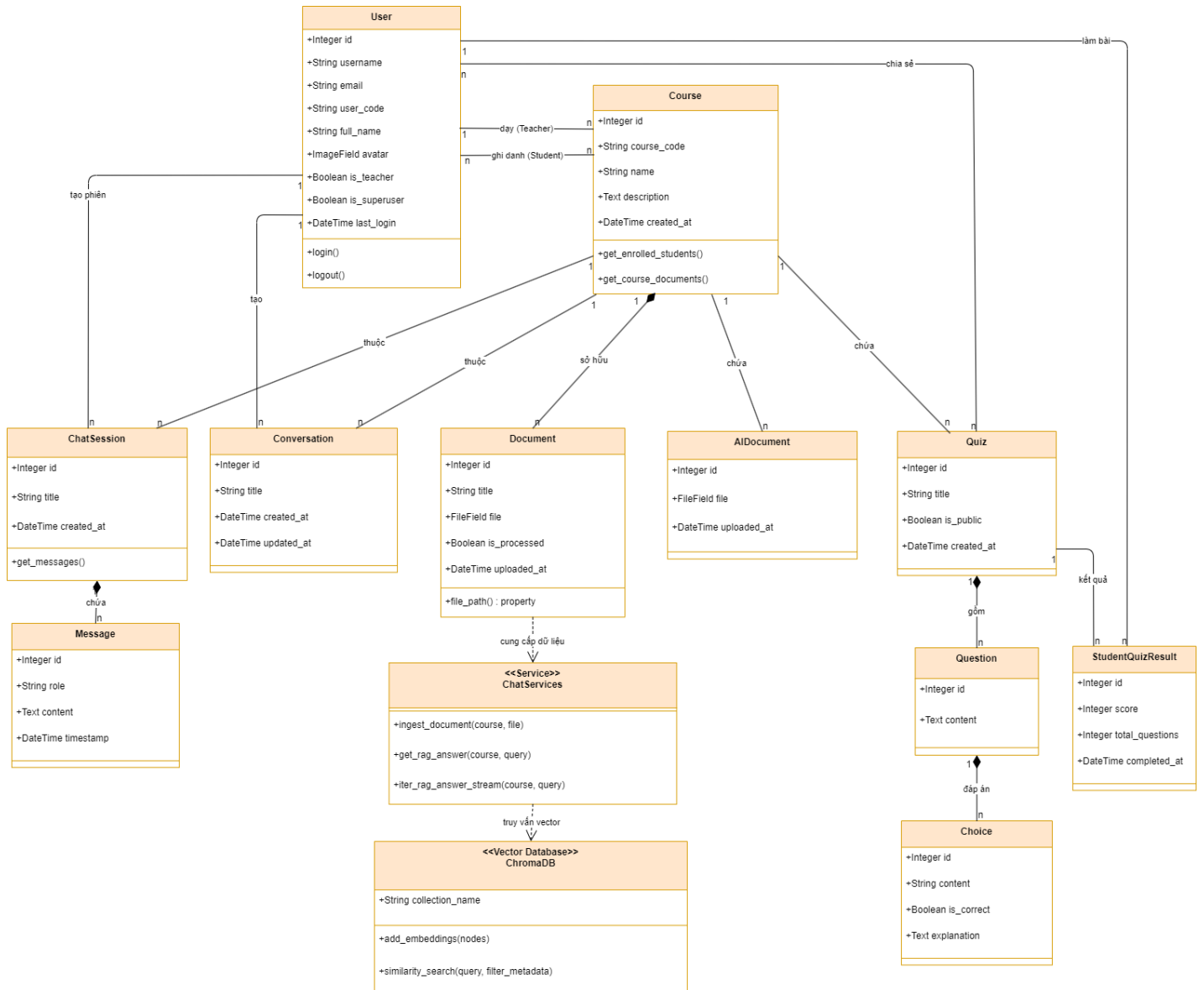
2.2.2 Tích hợp ChromaDB (Vector Database):

Cơ sở dữ liệu quan hệ không phù hợp để tìm kiếm trong các đoạn văn bản dài và phi cấu trúc như nội dung PDF. Vì vậy, hệ thống sử dụng ChromaDB để lưu trữ vector.

Quy trình hoạt động: nội dung tài liệu được mô hình Gemini Embedding 001 chuyển đổi thành các vector nhiều chiều, sau đó lưu vào ChromaDB. Mỗi môn học được cấp một

collection riêng, đặt tên theo mã môn học để tránh nhầm lẫn dữ liệu. Khi sinh viên đặt câu hỏi, hệ thống tìm đoạn văn bản phù hợp nhất dựa trên Cosine Similarity.

2.2.3 Sơ đồ Lớp:



Hình 3 Class Diagram

Sơ đồ lớp mô tả cấu trúc các thành phần chính trong mã nguồn backend:

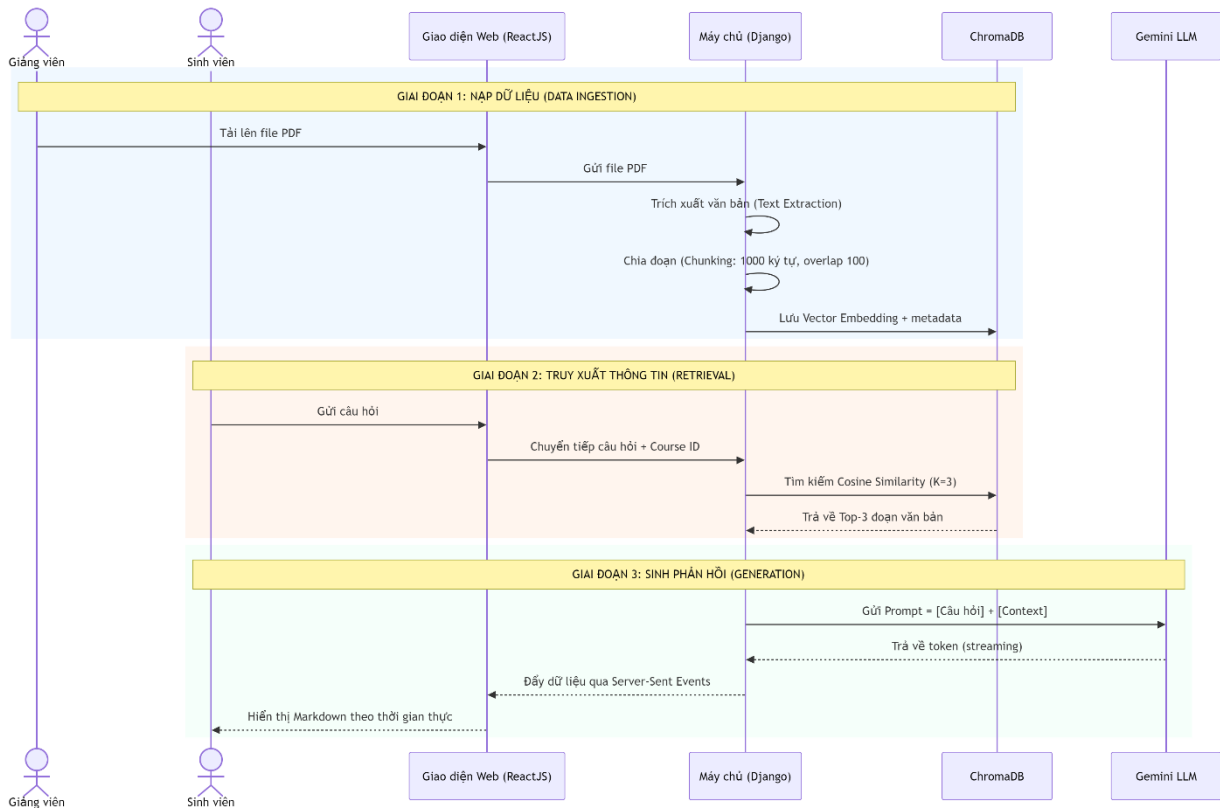
- Nhóm thực thể dữ liệu: User lưu thông tin người dùng, Course lưu danh mục môn học, Document lưu tài liệu PDF. Giảng viên quản lý môn học qua quan hệ 1-N, sinh viên ghi danh qua quan hệ N-N.
- Nhóm hội thoại: Conversation và ChatSession quản lý phiên chat theo từng user và từng môn. Message lưu nội dung trao đổi trong phiên, phân biệt lượt hỏi (user) và lượt trả lời (ai). AIDocument lưu tài liệu tri thức AI riêng theo môn.
- Nhóm trắc nghiệm: Quiz chứa danh sách Question, mỗi Question có nhiều Choice. StudentQuizResult ghi lại điểm số khi sinh viên hoàn thành bài thi.
- Nhóm xử lý RAG: ChatServices là lớp dịch vụ xử lý logic chatbot nhận câu hỏi, gọi ChromaDB tìm đoạn văn bản liên quan, rồi gửi cho Gemini sinh câu trả lời. ChromaDB lưu trữ vector và thực hiện tìm kiếm tương đồng.

CHƯƠNG 3 KIẾN TRÚC HỆ THỐNG VÀ CÔNG NGHỆ RAG

3.1 Sơ đồ luồng dữ liệu RAG:

Để đảm bảo AI chỉ trả lời dựa trên tài liệu do giảng viên cung cấp, hệ thống xây dựng một pipeline khép kín gồm 3 giai đoạn: Nạp dữ liệu $\Rightarrow$ Truy xuất $\Rightarrow$ Sinh phản hồi.

Dưới đây là sơ đồ tuần tự (Sequence Diagram) biểu diễn lớp kiến trúc luồng dữ liệu RAG được thiết kế và vận hành trong dự án:



Hình 4 Sequence Diagram

Giai đoạn 1) Nạp dữ liệu: Giảng viên tải file PDF lên hệ thống. Back-end bóc tách văn bản, chia nhỏ nội dung thành các đoạn theo cấu hình 1000 ký tự, overlap 100 ký tự. Các đoạn này được chuyển đổi thành vector và lưu vào ChromaDB kèm metadata.

Giai đoạn 2) Truy xuất: Khi sinh viên đặt câu hỏi, hệ thống chuyển câu hỏi thành vector rồi tìm kiếm các đoạn văn bản tương đồng nhất trong ChromaDB bằng Cosine Similarity, trả về K kết quả phù hợp nhất. Hệ thống cấu hình K=3, tức là lấy 3 đoạn văn bản tương đồng nhất để đưa vào prompt đủ để cung cấp ngữ cảnh mà không làm prompt quá dài.

Giai đoạn 3) Sinh phản hồi: Back-end ghép câu hỏi của sinh viên cùng các đoạn văn bản vừa truy xuất vào một prompt, gửi cho Gemini xử lý. Kết quả được trả về dạng stream

qua Server-Sent Events và hiển thị trực tiếp lên giao diện dưới dạng Markdown theo thời gian thực.

3.2 Cơ chế phân vùng dữ liệu:

Một rủi ro phổ biến khi dùng vector chung là dữ liệu của môn học này bị lẫn sang môn khác. Ví dụ: sinh viên hỏi trong môn Tin học nhưng hệ thống lại trả kết quả từ tài liệu môn Toán vì nội dung có từ khóa tương tự.

Để khắc phục, mỗi môn học được lưu trong một collection riêng trong ChromaDB, đặt tên theo mã môn học:

```
vectorstore = Chroma(  
    persist_directory=VECTOR_DB_PATH,  
    collection_name=f"course_{course_code}"  
)
```

Nhờ vậy, mỗi câu hỏi chỉ tìm kiếm đúng trong vùng dữ liệu của môn học đó, vừa tránh nhầm lẫn, vừa tăng tốc độ tìm kiếm do không gian vector được thu hẹp. Ngoài ra, hệ thống còn áp dụng bộ lọc quyền riêng tư ở cấp tài liệu, chỉ cho phép truy xuất các tài liệu công khai hoặc thuộc về chính người dùng đó:

```
search_filter = { "$or": [ {"is_public": True}, {"user_id": str(user_id)} ] }
```

CHƯƠNG 4 GIẢI THÍCH KỸ THUẬT CHUYÊN SÂU

4.1 Kỹ thuật phân đoạn tài liệu:

LLM và vector database đều có giới hạn về độ dài văn bản có thể xử lý trong một lần. Vì vậy, không thể nạp nguyên một file PDF dài vào hệ thống mà cần chia nhỏ nội dung thành các đoạn trước.

Hệ thống sử dụng RecursiveCharacterTextSplitter của LangChain với hai tham số chính:

1. `chunk_size = 1000`: Mỗi đoạn tối đa 1000 ký tự. Nếu quá nhỏ, đoạn văn bị cắt vụn, mất ngữ nghĩa. Nếu quá lớn, một đoạn chứa nhiều chủ đề khác nhau, làm giảm độ chính xác khi tìm kiếm.
2. `chunk_overlap = 100`: 100 ký tự cuối của đoạn trước được lặp lại ở đầu đoạn sau, tránh trường hợp một câu bị cắt đứt giữa chừng và duy trì liên kết ngữ nghĩa giữa các đoạn.

4.2 Thiết kế Prompt để ngăn chặn Hallucination:

Để AI hoạt động đúng vai trò trợ giảng thay vì tự do sáng tạo thông tin, hệ thống nhúng sẵn một System Prompt cố định vào Back-end với các quy tắc ràng buộc:

1. CHỈ SỬ DỤNG thông tin từ các đoạn trích này để trả lời.
2. Những đoạn trích KHÔNG liên quan đến câu hỏi thì PHÓT LỖ hoàn toàn.
3. Nếu không có đoạn trích nào chứa câu trả lời, hãy nói: "Dạ em chưa tìm thấy thông tin này trong tài liệu môn học ạ."
4. NẾU CÓ TRẢ LỜI, BẮT BUỘC phải liệt kê những tài liệu mà bạn THỰC SỰ ĐÃ DÙNG ở cuối câu trả lời theo đúng định dạng sau:

Nguồn tham khảo:

- [Tên file] (Trang [Số trang])

Phân tích từng quy tắc:

- Quy tắc 1 & 2: AI chỉ được dùng đúng tài liệu được cung cấp, không được tự bịa thêm thông tin từ dữ liệu huấn luyện gốc.
- Quy tắc 3: Khi câu hỏi không liên quan đến tài liệu, AI từ chối thay vì cố tạo ra câu trả lời.
- Quy tắc 4: Mỗi câu trả lời đều kèm tên file và số trang, giúp sinh viên kiểm chứng và tạo độ tin cậy với giảng viên.

4.3 Phản hồi thời gian thực bằng Server-Sent Events:

Với REST API thông thường, client phải chờ LLM xử lý xong toàn bộ câu trả lời rồi mới nhận kết quả, có thể mất hơn 15 giây với nội dung dài. Để khắc phục, hệ thống dùng SSE, một kết nối một chiều từ server đến client. Back-end Django sử dụng Generator để đẩy từng token ra ngoài ngay khi Gemini vừa tạo ra, thay vì chờ hoàn tất. Kết quả là giao diện hiển thị chữ chạy dần như chatbot thực thụ, với độ trễ ban đầu thấp.

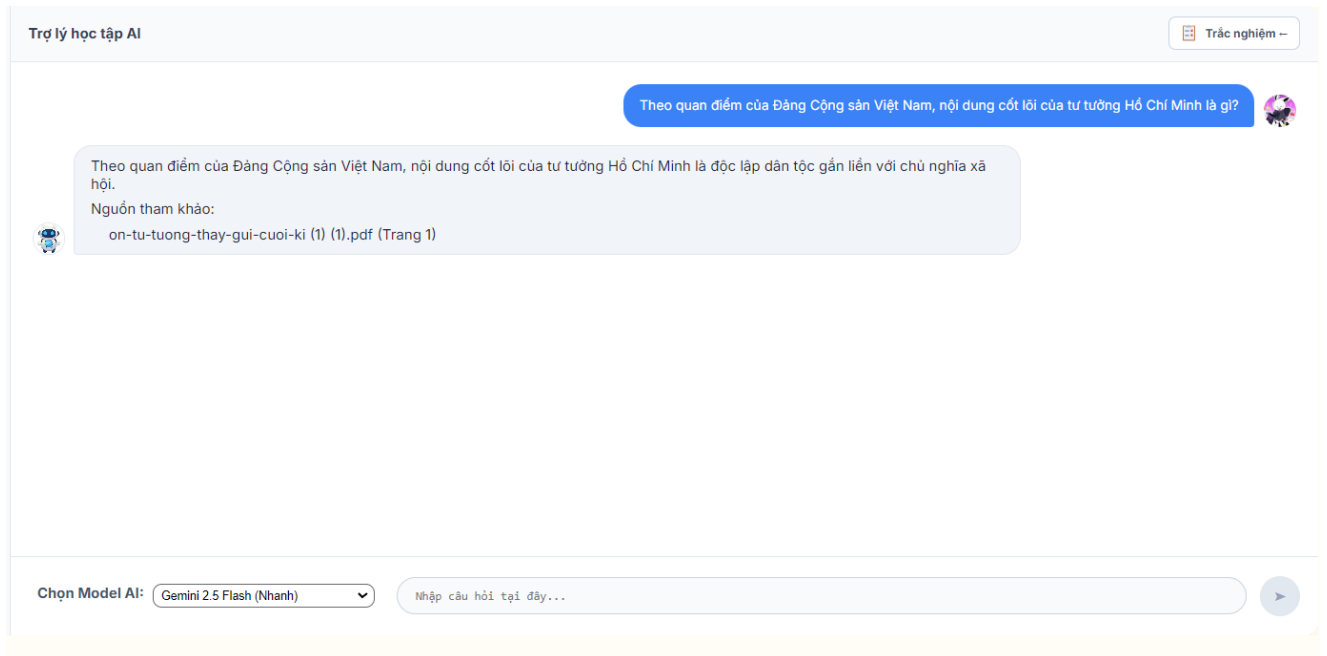
4.4 Thực chứng kiểm thử Ảo giác thông tin:

Nhóm tiến hành kiểm thử hộp đen với 2 kịch bản thực tế:

Kịch bản 1 - Câu hỏi đúng trọng tâm:

- Sinh viên hỏi: *"Theo quan điểm của Đảng Cộng sản Việt Nam, nội dung cốt lõi của tư tưởng Hồ Chí Minh là gì? "*
- Hệ thống: ChromaDB truy xuất thành công đoạn văn bản liên quan trong tài liệu ôn tập môn học.
- Chatbot Output: Trả về đáp án chính xác kèm theo trích dẫn nguồn tài liệu và số trang để sinh viên đối chiếu:

"Theo quan điểm của Đảng Cộng sản Việt Nam, nội dung cốt lõi của tư tưởng Hồ Chí Minh là độc lập dân tộc gắn liền với chủ nghĩa xã hội. Nguồn tham khảo: on-tu-tuong-thay-gui-cuoi-ki (1) (1).pdf (Trang 1)"

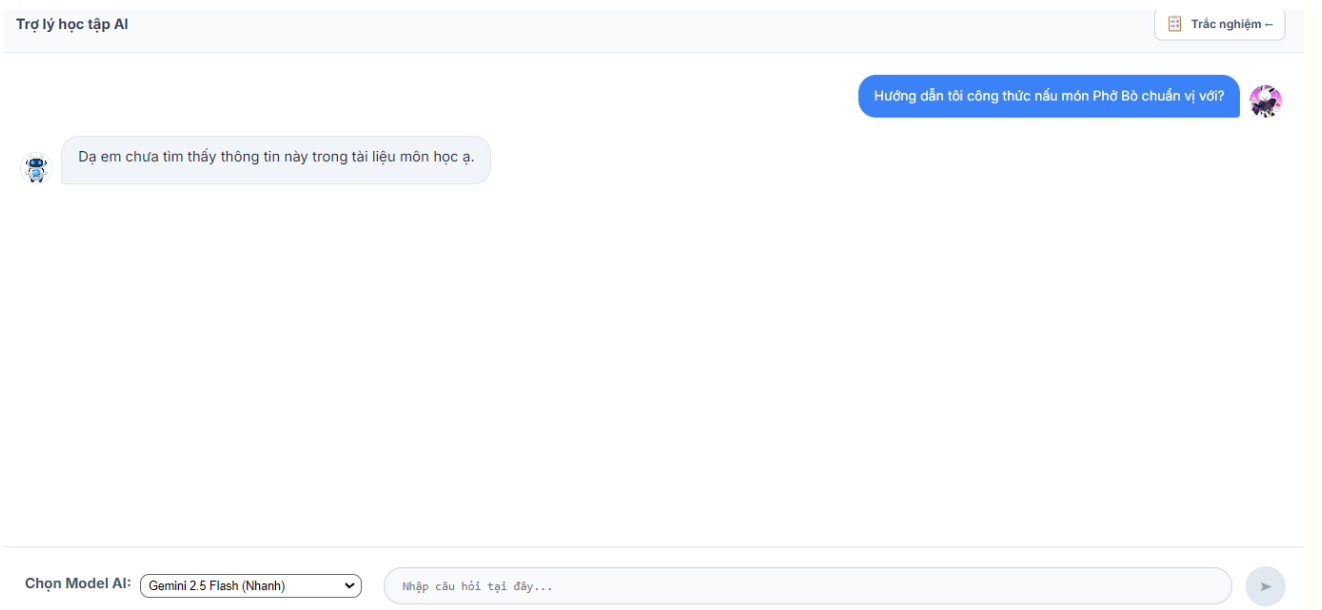


Hình 5 Giao diện Chatbot phản hồi kiến thức từ tài liệu môn học.

Kịch bản 2: Câu hỏi không liên quan/không có trong tài liệu:

- Sinh viên hỏi: *"Hướng dẫn tôi công thức nấu món Phở Bò chuẩn vị với?"*
- Hệ thống: ChromaDB tìm kiếm nhưng không có đoạn nào có độ tương đồng đủ cao. Kết quả trả về rỗng.
- Chatbot trả lời: Kích hoạt quy tắc từ chối, không bịa thông tin:

"Dạ em chưa tìm thấy thông tin này trong tài liệu môn học ạ."

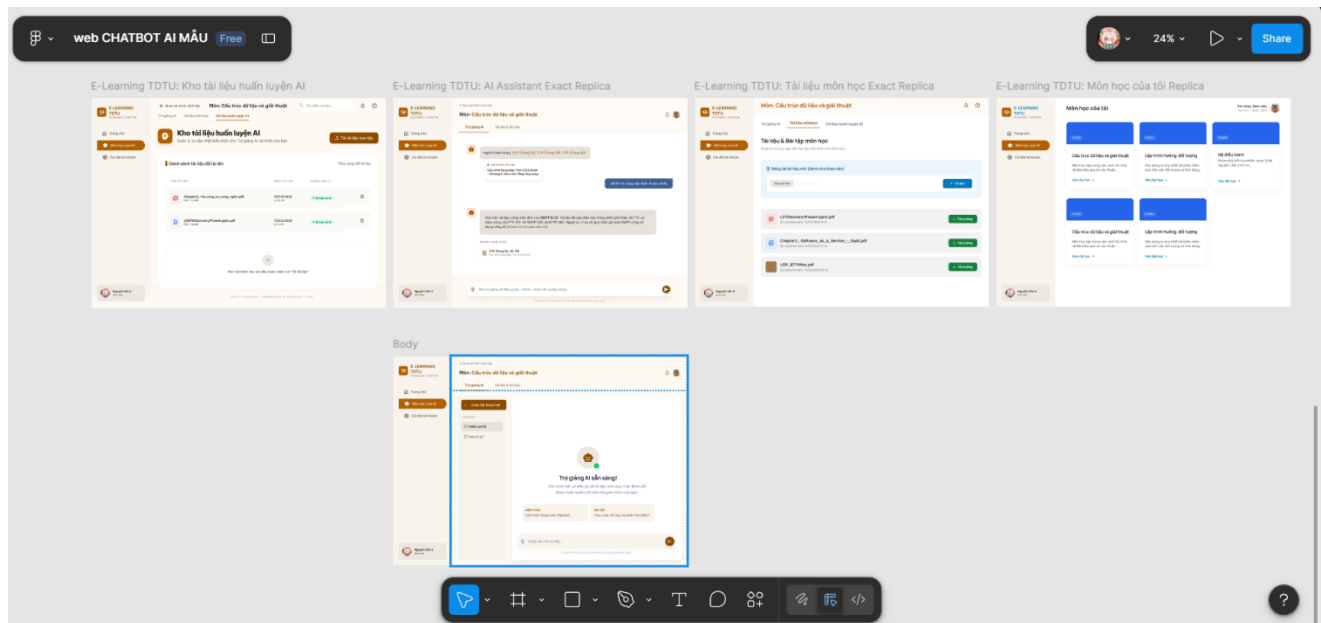


Hình 6 Giao diện Chatbot phản hồi Câu hỏi không liên quan/không có trong tài liệu

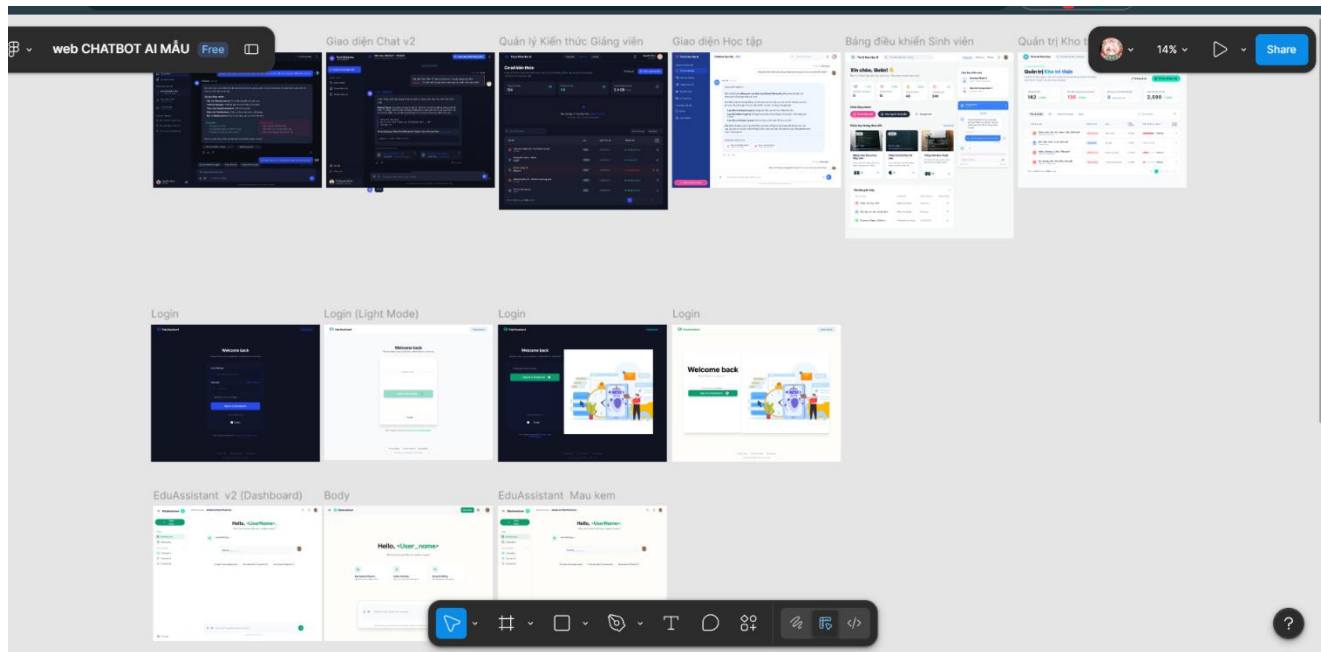
CHƯƠNG 5 GIAO DIỆN VÀ TRẢI NGHIỆM TƯƠNG TÁC

Một hệ thống tốt không chỉ cần xử lý dữ liệu hiệu quả ở back-end mà còn phải có giao diện thân thiện, dễ dùng. Nhóm thiết kế giao diện theo triết lý lấy người dùng làm trung tâm.

Trước khi lập trình giao diện, nhóm thiết kế prototype trên Figma để thống nhất bố cục, màu sắc và luồng điều hướng giữa các thành viên. Việc có bản mẫu từ trước giúp rút ngắn thời gian chỉnh sửa trong quá trình code và đảm bảo giao diện cuối nhất quán với định hướng ban đầu. Các màn hình được thiết kế bao gồm Dashboard, màn hình Chat, trang cài đặt tài khoản và một số màn hình khác....



Hình 7 Một số mẫu Giao diện Figma đã triển khai(1)



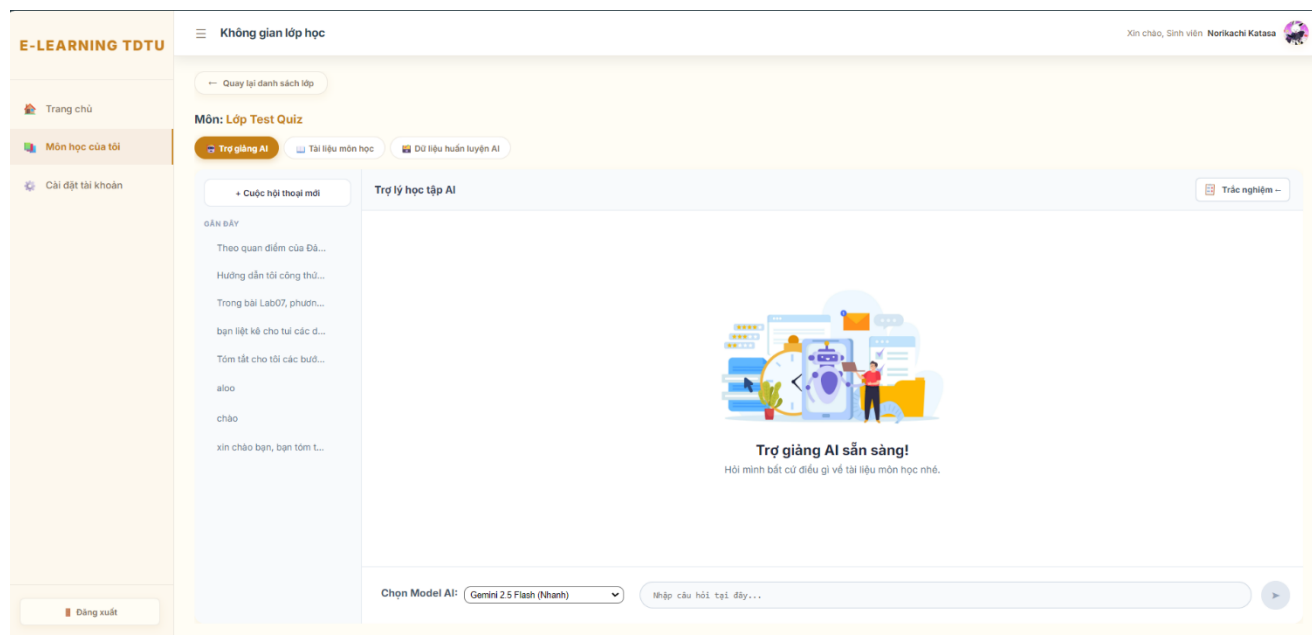
Hình 8 Một số mẫu Giao diện Figma đã triển khai(2)

5.1 Hiện thị nội dung chatbot:

Toàn bộ câu trả lời từ Chatbot được render theo chuẩn Markdown trên giao diện. Hệ thống hỗ trợ hiển thị đầy đủ các định dạng bao gồm đoạn code (code block), chữ in đậm, danh sách có thứ tự và không thứ tự, giúp các nội dung học thuật chuyên ngành được trình bày rõ ràng, dễ đọc và trực quan hơn so với việc hiển thị văn bản thuần.

5.2 Bố cục toàn màn hình:

Nhiều dự án web thường giới hạn chiều rộng giao diện ở mức cố định, khiến màn hình lớn bị thừa khoảng trống hai bên và nội dung bị co cụm. Nhóm áp dụng bố cục mở theo phong cách các nền tảng AI hiện đại như ChatGPT hay Gemini, cho phép khung hội thoại tự động co giãn theo kích thước trình duyệt. Điều này đặc biệt quan trọng với chatbot học thuật vì các đoạn code, bảng biểu và danh sách thuật ngữ dài cần không gian hiển thị rộng để không bị vỡ chữ.

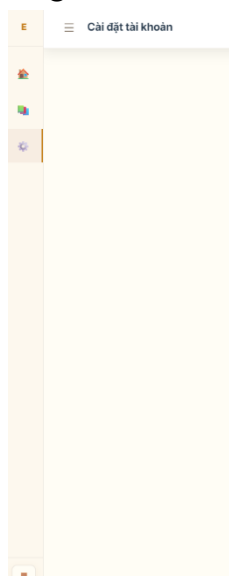


Hình 9 Giao diện bố cục mở rộng

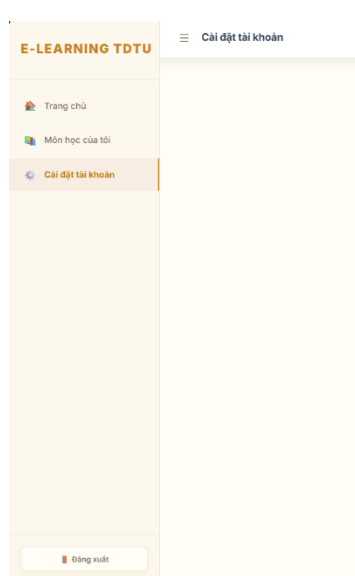
5.3 Cơ chế thanh công cụ động:

Khu vực quan trọng nhất với sinh viên là cửa sổ chat ở giữa. Để tối đa hóa không gian chat mà không mất đi thanh điều hướng, nhóm xây dựng sidebar có thể thu gọn bằng State Management trong ReactJS.

Khi nhấn nút Hamburger Menu, sidebar không ẩn hoàn toàn mà thu về mép trái, chỉ hiển thị icon chức năng. Sinh viên vừa có thêm không gian chat, vừa có thể chuyển mục nhanh mà không cần mở lại sidebar.



Hình 10 Sidebar đóng



Hình 11 Sidebar Mở

5.4 Trang cài đặt dạng Split-View:

Trang cài đặt tài khoản được thiết kế theo kiểu chia đôi màn hình 50/50:

- Nửa trái: Form nhập thông tin cá nhân, tự động thay đổi theo vai trò người dùng, sinh viên thấy thêm trường "Mã số sinh viên", "Khoa/Viện"; giảng viên thấy trường "Khoa giảng dạy". Không hiển thị thông tin thừa.
- Nửa phải: Hình minh họa phong cách Flat Design, hòa trộn vào nền màu kem, tạo cảm giác liền mạch.

Cài đặt tài khoản

Hồ sơ của tôi



Norikachi Katasa

Sinh viên

Thông tin sinh viên bổ sung

Mã số sinh viên (MSSV)

VD: 52100001

Được dùng để định danh khi nộp bài.

Lớp sinh hoạt

VD: 20H1323

Khoa / Viện

VD: Khoa CNTT

Khoa chuyên ngành của bạn tại TDTU.

Lưu thay đổi



Cài đặt Trợ lý AI

Model AI mặc định

Chọn mô hình AI sẽ tự động kích hoạt khi bạn mở khung chat.

Gemini 2.5 Flash (Nhanh)

Xóa lịch sử trò chuyện

Xóa toàn bộ các phiên chat của bạn trên hệ thống.

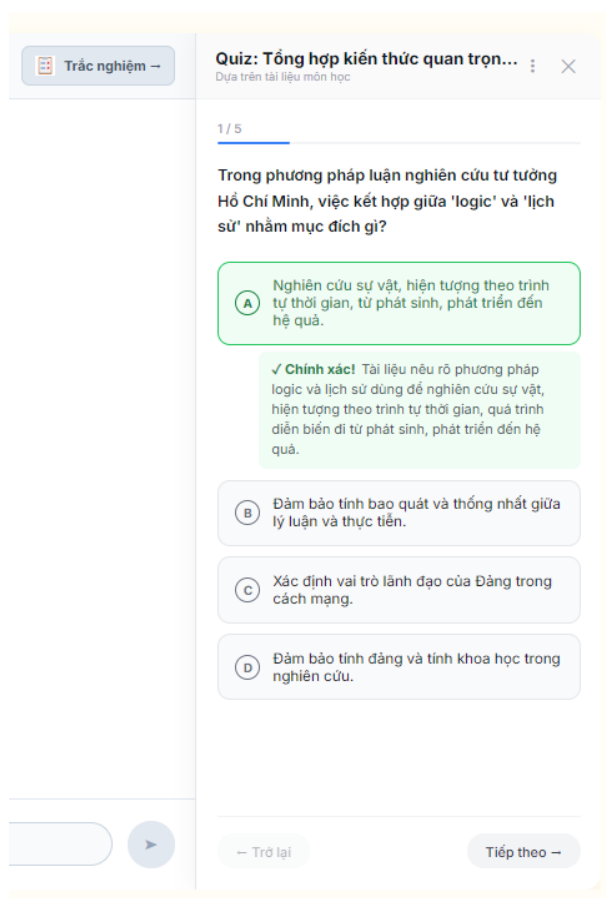
Xóa dữ liệu

Hình 12 Giao diện cài đặt người dùng.

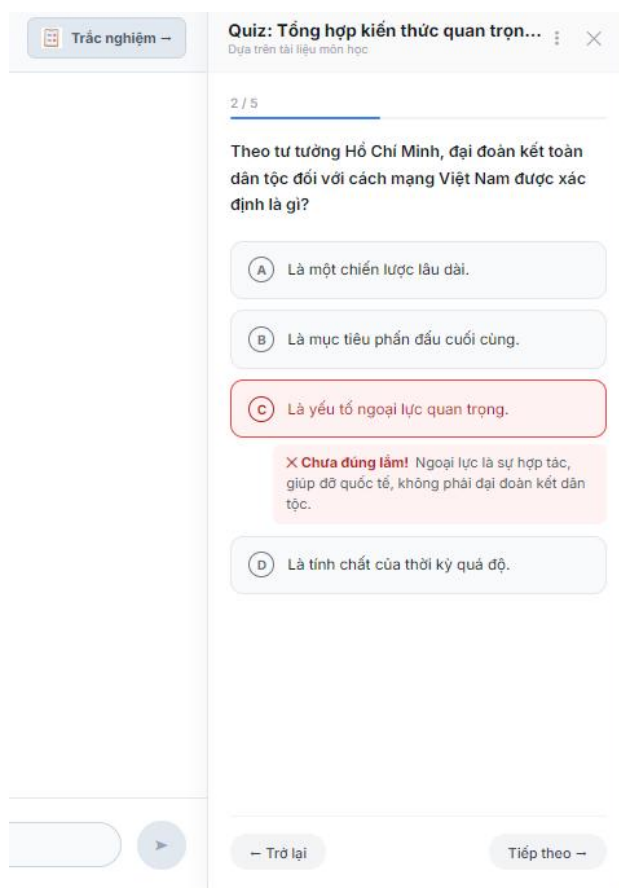
5.5 Hệ thống Trắc nghiệm bài tập:

Ngoài chatbot, hệ thống tích hợp thêm tính năng trắc nghiệm theo hướng game hóa để sinh viên ôn tập hiệu quả hơn.

Giao diện trắc nghiệm thiết kế dạng card bo góc, kết hợp hiệu ứng động cho bảng điểm. Khi sinh viên chọn đáp án, hệ thống phản hồi ngay bằng màu sắc trực quan, xanh lá nếu đúng, viền đỏ nếu sai thay vì chỉ hiện thông báo chữ. Cách phản hồi này giúp giảm áp lực học tập và tăng khả năng ghi nhớ kiến thức hơn cho sinh viên.



Hình 14 Giao diện chức năng luyện tập trắc nghiệm (đúng)



Hình 13 Giao diện chức năng luyện tập trắc nghiệm (sai).

CHƯƠNG 6 TRIỂN KHAI VÀ ĐÓNG GÓI HỆ THỐNG BẰNG DOCKER

6.1 Container hóa môi trường:

Khi chuyển giao dự án giữa các thiết bị khác nhau, lập trình viên thường gặp phải tình trạng ứng dụng chạy tốt trên máy cá nhân nhưng lại phát sinh lỗi trên máy khác do sự sai lệch về phiên bản thư viện hoặc hệ điều hành. Hệ thống của nhóm lại sử dụng song song hai nền tảng là Node.js cho Frontend và Python cho Backend. Để giải quyết triệt để rủi ro xung đột môi trường, nhóm đã ứng dụng Docker để đóng gói từng thành phần vào các Container độc lập. Mỗi Container sẽ tự quản lý tài nguyên và thư viện riêng, đảm bảo ứng dụng vận hành ổn định trên mọi máy chủ.

6.2 Cấu hình mạng nội bộ với Docker Compose:

Thay vì phải khởi động lẻ tẻ từng Container, hệ thống sử dụng tệp lệnh “docker-compose.yml” để quản lý tập trung. Điểm mấu chốt ở bước này là việc thiết lập Bridge Network để tạo ra một mạng nội bộ ảo. Nhờ đó, Backend Django có thể kết nối thẳng với cơ sở dữ liệu ChromaDB thông qua tên định danh nội bộ thay vì dùng địa chỉ IP. Cách làm này giúp kết nối luôn ổn định, không bị ảnh hưởng bởi tường lửa hay tình trạng dải mạng IP thay đổi liên tục.

6.3 Khởi chạy toàn bộ hệ thống bằng một lệnh duy nhất:

Toàn bộ các công đoạn phức tạp bao gồm kéo Image, tải Node modules, build ReactJS, cài đặt thư viện Python, nạp dữ liệu Database và khởi động Server đều được tự động hóa hoàn toàn bằng một dòng lệnh duy nhất: `docker-compose up --build -d`

Trong đó, tham số `-d` giúp hệ thống chạy ngầm và không chiếm dụng cửa sổ dòng lệnh. Chỉ mất vài phút, một môi trường hoàn chỉnh đã sẵn sàng hoạt động. Để thuận tiện cho việc nghiệm thu, nhóm cũng đã đính kèm tệp hướng dẫn sử dụng ghi rõ các đường dẫn truy cập và danh sách tài khoản phân quyền sẵn.

CHƯƠNG 7 TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

7.1 Đánh giá kết quả ứng dụng

Dự án đã đáp ứng đầy đủ các tiêu chí ban đầu đề ra:

- Kiến trúc Full-Stack hoàn chỉnh. Kết hợp thành công giao diện Frontend ReactJS và hệ thống máy chủ Backend Django.
- Vận hành trơn tru luồng RAG, xử lý tự động toàn bộ quy trình từ bóc tách văn bản, chia nhỏ đoạn dữ liệu cho đến vector hóa tài liệu.
- Tối ưu trải nghiệm người dùng. Giao diện được thiết kế mở rộng dễ thao tác, tích hợp tính năng ôn tập trắc nghiệm sinh động và đặc biệt là khả năng phản hồi theo thời gian thực nhờ công nghệ kết nối SSE.
- Triển khai chuyên nghiệp. Mã nguồn được đóng gói gọn gàng bằng Docker, tiện lợi cho việc bàn giao và cài đặt.
- Qua quá trình kiểm thử với nhiều câu hỏi thuộc và ngoài phạm vi tài liệu, hệ thống phản hồi đúng ngữ cảnh trong các trường hợp câu hỏi nằm trong tài liệu và từ chối chính xác khi câu hỏi không liên quan đúng với thiết kế Prompt Engineering đề ra.

7.2 Hạn chế còn tồn đọng của hệ thống

Vấn đề lớn nhất của hệ thống hiện tại là tốc độ phản hồi của mô hình ngôn ngữ. Do ứng dụng đang chạy trên máy tính cá nhân không có card đồ họa chuyên dụng và phải phụ thuộc hoàn toàn vào đường truyền mạng khi gọi API của Google Gemini, hệ thống đôi khi sẽ bị trễ vài giây ở câu trả lời đầu tiên nếu mạng không ổn định.

7.3 Hướng phát triển chuyên môn sâu trong tương lai

Nếu có cơ hội đưa dự án vào ứng dụng thực tế, nhóm dự định sẽ nâng cấp thêm hai tính năng cốt lõi:

- Hệ thống tìm kiếm lai: Kết hợp thuật toán tìm kiếm ngữ nghĩa Vector với cơ chế tìm kiếm từ khóa chuẩn xác. Giải pháp này sẽ khắc phục triệt để hạn chế của AI khi sinh viên cần tra cứu các con số, mã luật hoặc ký hiệu chuyên ngành (ví dụ như "Khoản 3, Mục A, Nghị định 100").
- Tự động phác họa Sơ đồ tư duy: Tinh chỉnh câu lệnh để AI trả về song song văn bản và cấu trúc dữ liệu JSON. Từ đó, hệ thống có thể tự động vẽ biểu đồ trực quan ngay trên giao diện web, giúp sinh viên nắm bắt bài học nhanh chóng hơn.

PHỤ LỤC: BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM

STT	Sinh viên	Công việc	Hoàn thành
1	Nguyễn Văn Phát (MSSV: 52300049)	- Frontend: Thiết kế prototype Figma, xây dựng cấu trúc ReactJS, Expansive Layout. Xây dựng giao diện Dashboard Admin quản lý người dùng, môn học và sinh viên. - Backend: Xây dựng API Quản lý Tài liệu (Knowledge Base) cho phép upload và quản lý file. Tích hợp tính năng phụ trợ Tạo bài tập trắc nghiệm tự động (Quiz Generator). - Tài liệu: soạn thảo Chương 2, Chương 6, Chương 7; tổng hợp và định dạng toàn bộ báo cáo.	100%
2	Hoàng Minh Quân (MSSV: 52300245)	- Frontend: Xây dựng giao diện làm bài trắc nghiệm và tích hợp hiển thị định dạng Markdown cho khung chat. - Backend: Cấu hình Authentication (Đăng nhập hệ thống). Xây dựng tính năng Streaming Response bằng Server-Sent Events (SSE), Quản lý ngữ cảnh hội thoại (Memory) và thiết lập Tùy chọn Model AI. - Tài liệu: soạn thảo Chương 3 và Chương 4, hỗ trợ kiểm thử và chụp ảnh giao diện minh họa cho báo cáo.	100%
3	Nguyễn Thành Thắng (MSSV: 52300256)	- Frontend: Xây dựng Giao diện Chatbot và thiết kế trang xem lại Lịch sử hội thoại dạng Split-View. - Backend & DevOps: Xây dựng luồng RAG cốt lõi bao gồm Xử lý dữ liệu (Parsing, Chunking, Embedding), Truy vấn VectorDB và Trích dẫn nguồn (Source Citation). Đóng gói toàn bộ hệ thống bằng Docker Compose. - Tài liệu: soạn thảo Chương 1 và Chương 5, và soạn thảo tài liệu hướng dẫn sử dụng.	100%

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Anh:

- ChromaDB Core Architecture
TryChroma <https://docs.trychroma.com/getting-started>
- LangChain Conceptual Guide (Python) -
LangChain <https://python.langchain.com/docs/concepts/>
- Gemini API Documentation
Google AI for Developers <https://ai.google.dev/docs>
- Vite and React Frontend Tooling
ViteJS <https://vitejs.dev/guide/>
- Django ORM and Database Management
Django Software Foundation <https://docs.djangoproject.com/en/stable/topics/db/>
- Docker-Compose Networking
Docker Docs <https://docs.docker.com/compose/networking/>

Tài liệu tiếng Việt:

- Triển khai ứng dụng linh hoạt với cơ chế Containerization qua Docker - HowKteam
<https://howkteam.vn/course/su-dung-docker/gioi-thieu-ve-docker-4158> HowKteam